

Beyond "Taste": The New Code Quality Moat in the AI Era

Introduction: From Taste to Design

A recent comment on a pull request made by a generative AI suggested the code was rejected due to **"taste"** – implying an experienced reviewer's intuitive sense of quality. Some have even claimed that *"taste is the new moat"* as AI makes coding easier ¹. In other words, when AI can generate passable code or designs on demand, the real competitive advantage shifts to human judgment, restraint, and knowing what actually matters. But calling this **"taste"** only scratches the surface. What seasoned engineers and architects bring is more concrete: a strategic **design sense**, understanding of long-term implications, and alignment with non-functional requirements like security and performance. It's not just about aesthetic preference or personal style – it's about design quality and foresight. This document explores why **code quality in the AI era is about much more than taste**, and how human insight remains the critical factor for maintaining a robust, sustainable codebase.

Beyond Does It Work: Code as Long-Term Design

In the past, code reviews often centered on *"Does it work? Does it pass the tests?"* If yes, it might get approved. But experienced developers know that every new line of code has ripple effects. **Each addition can either improve the codebase's overall architecture or introduce new technical debt.** This mental calculation – *"Will this change make the system better in the long run or harder to maintain?"* – is an integral part of a senior engineer's review process. It's sometimes dubbed *taste*, but in reality it's a blend of design and architecture thinking.

Generative AI doesn't (yet) grasp these nuances. A recent report found that **AI-generated code tends to be highly functional "but systematically lacking in architectural judgment"** ². The AI will churn out a solution that technically works, yet it may follow textbook patterns too rigidly or ignore context, resulting in designs that aren't optimal for the specific application ³. For example, AI often **over-specifies solutions**, coding for extreme edge cases that a human reviewer might intentionally omit or defer for simplicity ⁴. This highlights a key difference: a human designer makes *strategic decisions* about what *not* to include, aiming to keep the solution as simple and elegant as possible for the problem at hand.

In essence, what some call *"taste"* is really this **design-driven judgment**. It's the ability to envision how a piece of code will age within the system – will it make future changes easier or harder? Does it introduce subtle performance or security issues? Seasoned reviewers apply unwritten rules and experiences to assess these factors, even if they can't always quantify them. They might say *"this code isn't quite right"* not out of personal taste in a vacuous sense, but because they sense misalignment with the system's architecture or future direction.

Code Quality as the New Moat

As AI coding tools proliferate, *"if it works, ship it"* is no longer a sufficient standard. When **anyone can generate a working snippet**, the true differentiator becomes **code quality and design philosophy**.

⁵ . One industry observer noted that if “*anyone can implement, raw implementation loses power to differentiate*” – instead, competitive advantage shifts to higher-level concerns like **what** to build and **how** it should behave, which they term “*design ethos*” or taste ⁵ ⁶ . In software, this design ethos includes architecture, scalability, clarity, and maintainability.

Let’s clarify the term “taste.” In the context of AI-generated output, **taste is essentially the human ability to curate and refine** what the AI produces ⁷ ⁸ . It’s comparable to a photographer’s eye in the age of ubiquitous camera phones – the technology can produce a perfectly exposed image, but the artistic judgment of what to capture and what to leave out remains with the human. In coding, this means having an eye for elegant abstraction, consistency with project conventions, and forward-looking design. It’s making choices about code structure and naming that ensure the code not only works today but remains readable and adaptable next year.

Crucially, **taste in coding involves knowing when *not* to add something**. A well-known maxim by Antoine de Saint-Exupéry says that “*perfection is achieved, not when there is nothing more to add, but when there is nothing left to take away.*” ⁹ Good engineers live by this principle. They resist the temptation to shove in every possible feature or fix “**just in case,**” focusing instead on what truly needs to be there. This disciplined minimalism results in cleaner, more robust code. AI, on the other hand, has no innate sense of YAGNI (“You Aren’t Gonna Need It”) – it will cheerfully generate whatever functionality its instructions imply, even if a human would judge that it’s overkill.

The AI Era: Code is Cheap, Design is Precious

One fascinating shift with AI-generated code is that **code itself is becoming a cheap commodity** – easy to create, throw away, and recreate in multiple variants. In traditional development, asking a teammate to significantly rework or rewrite code they labored on could be costly (in time and morale). As Speaker 1 noted, “*if you go to somebody who’s been working on something for a week and then say, ‘Great, now can you rewrite it like this, this, and this,’ they’d go crazy.*” Because of that human factor, teams often **compromised** and accepted code that was “good enough” functionally, even if not ideal in design.

With AI, that dynamic changes. We can be pickier. If a code generation doesn’t meet the quality bar, we can prompt the AI to **refine the code repeatedly without hurt feelings or burnout**. This means we no longer have to compromise on aspects like clarity, style, or adhering to architectural patterns. In theory, we can keep iterating until the code isn’t just working, but clean and well-designed. In practice, this raises our expectations for pull requests: does the code *not only* do the job, but do it in a way that fits our project’s long-term goals?

However, this optimistic view comes with a caveat. The ease of producing code can flood a codebase with *too much, too fast*. **Quantity is no substitute for quality**. There are already reports of AI-assisted developers unintentionally accelerating the accumulation of technical debt. One study by Ox Security describes AI coding tools as an “*Army of Juniors*” – like a horde of eager junior devs who **produce lots of code quickly but lack architectural wisdom and security awareness** ¹⁰ . Such code often **violates fundamental software engineering best practices** in ways a seasoned developer would catch ¹¹ . In other words, without strong human oversight, AI can create a massive volume of *working* code that is *architecturally unsound or fragile*.

Indeed, AI code generators tend to follow instructions to the letter, without the intuition to manage scope. They might reimplement similar code in multiple places rather than refactor (a human knows when to abstract functionality) ¹² . They may also introduce subtle bugs or security issues by blindly applying patterns, since **AI lacks the contextual judgment to know when a textbook solution**

doesn't apply. As one report put it, **AI is great at implementation, but human creativity and judgment are irreplaceable for higher-level decisions** ¹³. Successful teams are adapting by treating AI as a powerful assistant: the AI cranks out the bulk of the code, while human developers focus on **verification, design adjustments, and ensuring all the "ilities" (scalability, security, maintainability) are met** ¹⁴.

Knowing When to Stop: The Art of Restraint

A key element of software design "taste" is knowing when to stop coding. This might sound counterintuitive – why would stopping be hard when the AI can always generate more? But **restraint is a virtue that separates good design from over-engineered sludge**. Hollywood director Ben Affleck recently commented on AI in the creative arts, saying: *"Craft is knowing how to work; art is knowing when to stop."* Knowing when to stop, he argued, will be *"very difficult for AI to learn"* because it ties into human taste and judgment ¹⁵. In software, *artful* code often comes from cutting unnecessary complexity.

An AI doesn't inherently know when it has written *enough* code or when a solution should be simpler. Given a prompt, it will keep expanding or adding detail as long as connections in its training data allow. Humans, by contrast, can weigh trade-offs like **complexity vs. benefit**. For example, a human reviewer might decide that handling a rare edge-case isn't worth the extra 50 lines of convoluted logic it adds – those 50 lines would do more harm (in readability and bug surface area) than the rare bug they prevent. A human with good taste/design sense will say, *"Let's leave that case out for now,"* whereas an AI might have included it because it **could**.

This aspect of judgment also relates to the idea that **"everything has to earn its place."** In well-designed systems, every function, class, and line of code should justify its existence. If something isn't pulling its weight (or its purpose can't be clearly explained), it probably doesn't belong in the codebase. Seasoned architects often perform this mental audit during code reviews: *Is this new module truly necessary? Is there a simpler way to achieve the same result?* By trimming the excess, they keep the codebase lean and focused. Such **discretion is hard to encode into an algorithm**. AI might generate a perfectly correct module that solves a problem, but a human with a holistic view might realize that the problem didn't need solving *at that level of complexity*, or at all.

Bridging Engineering and Design in Code Reviews

Interestingly, as code generation becomes commoditized, software development begins to overlap more with classic design disciplines. The comment about "taste" hints that reviewing code might soon resemble reviewing a piece of design or art. This opens the door to involving **designers or professionals from humanities** in the code review process – people who excel at considering human-centric aspects, clarity of intent, and long-term ramifications. They might spot inconsistencies or unnecessary elements in a code change much like a UX designer critiques an interface for extra buttons that don't add value.

Consider the principle of **form follows function** in design: every element should serve a purpose and harmonize with the whole. In code, we similarly want the *form* (the code structure) to follow the *function* (the feature's intent) cleanly. Code can have an *aesthetic* in how clearly it expresses its intent. If a chunk of code feels clunky or overly convoluted relative to what it achieves, a design-minded reviewer will flag that. They might not use the term "ugly," but effectively that's what it is – an inelegant solution that could be refined.

Bringing in design thinking means asking questions like: **Does this code align with the product's vision and user needs?** Sometimes engineers implement a technically clever solution that isn't actually necessary for the user's goals. A product-minded perspective (often a strength of designers or product managers) ensures the code isn't just technically sound but also makes sense in the bigger picture. It's about having a **holistic vision**: seeing the code not as isolated tickets to close, but as part of a living system that should evolve coherently.

Non-Functional Requirements: The Invisible Guardians of Quality

"Taste," as redefined here, also encompasses diligently upholding non-functional requirements. These include **security, performance, scalability, readability, and maintainability** – qualities that aren't explicitly in the user story but are essential to a healthy software system. An AI given a task might not automatically consider, say, the security implications of a piece of code. If it outputs a working snippet that meets the functional spec, it considers the job done. A human reviewer with a design mindset will look deeper: *Does this code introduce a SQL injection risk? Will it scale poorly with large inputs? Is it using idiomatic patterns of the codebase or will others be confused reading it?*

There's evidence that **AI-generated code, if unchecked, can introduce security and maintenance risks**. The Ox Security analysis found multiple anti-patterns in AI-written code that could undermine software security ¹⁶ ¹³. For example, AI might produce code that technically works but isn't aligned with the system's intended architecture ("Return of Monoliths" was one anti-pattern noted, meaning an AI may ignorantly concentrate logic in one place where a human would have split it for modularity) ¹¹. Likewise, performance pitfalls might slip in – perhaps the AI writes an $O(n^2)$ solution that passes small test cases but would bog down in production. Human oversight is needed to catch these issues, ensuring that each change isn't just functionally correct, but also **secure, efficient, and maintainable**.

This is why some experts predict the **role of engineers will shift toward verification, oversight, and integration** rather than raw coding ¹⁷ ¹⁴. When AI handles the grunt work of writing code, the humans are freed (and required) to focus on whether the code *should* be written that way at all. In other words, the humans guard the **software's quality attributes and architectural vision**. Code reviews in the AI era will likely put much more weight on these aspects: not *"did the tests pass?"* but *"is this solution well-designed and aligned with our long-term needs?"*

Conclusion: Design and Judgment – The Lasting Moat

Generative AI has undeniably lowered the barrier for producing code that "works." In this new landscape, the idea that **"taste is the new moat"** captures a truth: human judgment – manifested through design sense, strategic thinking, and holistic quality awareness – is now the critical differentiator ⁵. But as we've explored, calling it *taste* might undersell its importance. It's really about **expertise in knowing what to build, how to build it properly, and when to say no**. It's about maintaining a vision for the codebase that extends beyond the immediate task or prompt.

Great software is as much about what you leave out as what you put in. The role of human developers is evolving to be more like curators and architects, ensuring that AI-generated contributions actually fit the grand design. They provide the "north star" of where the codebase should be heading and prune away anything that deviates or dilutes that vision. In this sense, **the new moat is design mastery and foresight** – the very human ability to make nuanced decisions that balance short-term needs with long-term consequences.

Moving forward, teams that recognize this shift will hold an advantage. They won't fear AI taking over coding, but will leverage it while doubling down on human strengths: creativity, critical thinking, and yes, *taste* – understood as the culmination of experience-driven insight. By raising the quality bar and refusing to accept mediocre-by-design code, we can harness AI's speed **without sacrificing the intentionality and excellence that truly great software demands** ¹⁴ ¹⁸ .

Ultimately, code is **"a codification of intent,"** as Speaker 1 put it. Our intents – the problems we choose to solve and how elegantly we solve them – define the value of our software. AI can help express those intents faster, but *deciding* the intent and sculpting the solution is a profoundly human art. That is the art of knowing when to stop, what to trim, and what to emphasize. And that art will remain our moat for the foreseeable future.

Sources:

- Takafumi Endo, *"Insights from Cursor: Beyond Token Limits and the Rise of Taste,"* Medium (2025) ⁵ ¹⁴ .
- Patrick Farry, *"AI-Generated Code Creates New Wave of Technical Debt, Report Finds,"* InfoQ (2025) ¹⁸ ¹³ .
- Ox Security, *"The Army of Juniors: The AI Code Security Crisis"* (2025) ¹⁰ ¹¹ .
- Allen Yao, *"Taste Over Tokens,"* Residual Thoughts Substack (2026) ⁸ .
- Ben Affleck's AI commentary (via LinkedIn transcript, 2024) ¹⁵ .
- Antoine de Saint-Exupéry, *Wind, Sand and Stars* (1939) ⁹ .

¹ ⁵ ⁶ ¹⁴ ¹⁷ Insights from Cursor: Beyond Token Limits and the Rise of Taste | by Takafumi Endo | Medium

<https://medium.com/@takafumi.endo/insights-from-cursor-beyond-token-limits-and-the-rise-of-taste-1e6ec2ec3396>

² ³ ⁴ ¹² ¹³ ¹⁶ ¹⁸ AI-Generated Code Creates New Wave of Technical Debt, Report Finds - InfoQ
<https://www.infoq.com/news/2025/11/ai-code-technical-debt/>

⁷ ⁸ Taste Over Tokens - by Allen Yao - Residual Thoughts
<https://getresidualthoughts.substack.com/p/taste-over-tokens>

⁹ Antoine de Saint Exupéry - Wikiquote
https://en.wikiquote.org/wiki/Antoine_de_Saint_Exup%C3%A9ry

¹⁰ ¹¹ Army of juniors report gated page | OX Security
<https://www.ox.security/resource-category/whitepapers-and-reports/army-of-juniors/>

¹⁵ "Craft is knowing how to work, art is knowing when to stop, and I think knowing when to stop is going to be a VERY difficult thing for AI to learn." Ben Affleck with a take on AI in the arts that's... | Michael Jackson | 53 comments
https://www.linkedin.com/posts/michaeljacksonvc_craft-is-knowing-how-to-work-art-is-knowing-activity-7264019180364914689-6V8T